

El siguiente es un extracto de código del programa de cómputo que se pretende registrar como obra:

Python

```
"""SessionController – LangGraph state machine for the diagnostic session
lifecycle.

Architecture invariants (Synapse_Arquitectura_Agentes_v1.0.md §10):
    D-IA-01: This module is the ONLY orchestrator of agent calls. No other module
        calls LLMs or transitions session states.

State machine:
    idle → interviewing → exploration_complete → audit_pending →
proposal_inputs_pending (optional)
    → audit_passed → proposals_ready

The transition graph is explicit and enforced – invalid transitions raise an
error.
All state is persisted to PostgreSQL (sessions table) after each transition.

LangGraph usage:
    - Each node in the graph corresponds to a handler function that processes
        a specific state and optionally transitions to the next one.
    - For Paso 3.1, nodes are stubs. The real logic is wired in Pasos 3.2–3.5.
    - The graph is compiled once at module level and reused across requests.
"""

from __future__ import annotations

import logging
import uuid
from collections.abc import Awaitable, Callable
from datetime import datetime, timezone
from typing import TYPE_CHECKING, Any

from langgraph.graph import END, StateGraph
from sqlalchemy.orm import Session

from app.db.models import Session as DBSession
from app.models.session import (
    AuditFinding,
    GraphDelta,
    IMO_COUNT,
    Message,
    MessageRole,
    SessionState,
    SessionStatus,
)

if TYPE_CHECKING:
    from app.models.session import GraphRestructureOutput
```

```

logger = logging.getLogger(__name__)

# — State transition graph (valid edges) —————

_VALID_TRANSITIONS: dict[SessionStatus, set[SessionStatus]] = {
    SessionStatus.idle: {SessionStatus.interviewing},
    SessionStatus.interviewing: {SessionStatus.exploration_complete},
    # exploration_complete: chat continues, canvas starts building in parallel.
    # Self-loop allowed because the interviewing node keeps setting this status
    # after IMOs stay at >=0.7. Only manual / future trigger moves to
    audit_pending.
    SessionStatus.exploration_complete: {SessionStatus.audit_pending,
SessionStatus.exploration_complete},
    SessionStatus.audit_pending: {
        SessionStatus.audit_passed,
        SessionStatus.proposal_inputs_pending,
    },
    SessionStatus.proposal_inputs_pending: {
        SessionStatus.audit_pending,
        SessionStatus.audit_passed,
    },
    SessionStatus.audit_passed: {SessionStatus.proposals_ready},
    SessionStatus.proposals_ready: set(), # terminal state
}

# Statuses where the interviewer is still active and can receive user messages.
# audit_pending and audit_passed are included because the current stubs
# auto-advanced some sessions there before the exploration_complete fix.
# Real gating to those states will happen when CausalAuditor is implemented (Paso
5.1).
_CHAT_ACTIVE_STATUSES: set[SessionStatus] = {
    SessionStatus.idle,
    SessionStatus.interviewing,
    SessionStatus.exploration_complete,
    SessionStatus.audit_pending,
    SessionStatus.proposal_inputs_pending,
    SessionStatus.audit_passed,
    # After proposals are generated, the Analyst can keep refining the graph
    # based on new user information – the session must remain conversational.
    SessionStatus.proposals_ready,
}

# — LangGraph node handlers (stubs for Paso 3.1) —————

def _node_idle(state: dict[str, Any]) -> dict[str, Any]:
    """Entry point – session awaits first user message."""
    logger.debug("Node: idle for session %s", state.get("session_id"))
    return state

```

```

async def _node_interviewing(state: dict[str, Any]) -> dict[str, Any]:
    """Node handler for the active problem mapping phase.

    Runs the analytical and conversational agents in parallel.
    When IMO coverage reaches thresholds, it transitions to exploration_complete.
    """
    import asyncio

    from app.agents.analyst import AnalystAgent
    from app.agents.interviewer import InterviewerAgent
    from app.core.context_manager import ContextManager
    from app.core.domain_classifier import DomainClassifier
    from app.core.interview_controller import ObjectiveTracker
    from app.models.session import Message as SessionMessage
    from app.models.session import SessionContext, Metric
    from app.providers import factory
    from app.providers.base import Message as ProviderMessage

    logger.debug("Node: interviewing for session %s", state.get("session_id"))

    provider = state.pop("__provider")
    context_manager = ContextManager()
    interviewer = InterviewerAgent(provider)
    analyst = AnalystAgent(provider)

    # Reconstruct objects for context manager
    transcript_objs = [SessionMessage(**m) for m in state.get("transcript", [])]
    last_turn = transcript_objs[-1] if transcript_objs else None

    # Analyst context needs preceding question + validated answer
    validated_turn: ProviderMessage | None = None
    if last_turn and last_turn.role == "user":
        validated_turn = ProviderMessage(role=last_turn.role,
        content=last_turn.content)

    # Parse what we have to SessionContext
    from app.models.session import FrameworkInfo, Metric, SessionStatus

    validated_metrics = [Metric(**m) for m in state.get("validated_metrics", [])]

    # Reconstruct FrameworkInfo from stored dict (populated on
    exploration_complete)
    _active_fw_raw = state.get("active_framework")
    _active_fw: FrameworkInfo | None = None
    if _active_fw_raw:
        try:
            _active_fw = FrameworkInfo(**(_active_fw_raw if
            isinstance(_active_fw_raw, dict) else _active_fw_raw.model_dump()))
        except Exception as _fw_parse_exc:

```

```

        logger.warning("Could not reconstruct FrameworkInfo from state: %s",
            _fw_parse_exc)

    ctx = SessionContext(
        session_id=state["session_id"],
        status=SessionStatus(state["status"]),
        transcript=transcript_objs,
        validated_metrics=validated_metrics,
        imo_scores=state.get("imo_scores", {}),
        graph_summary=None,
        analyst_output=state.get("analyst_output"),
        active_framework=_active_fw,
    )

    # — Build active solution context for Analyst awareness —————
    _solution_layers_raw = state.get("solution_layers")
    if _solution_layers_raw and isinstance(_solution_layers_raw, dict):
        _active_hz = _solution_layers_raw.get("active_horizon")
        if _active_hz:
            _layer_data = (_solution_layers_raw.get("layers") or
                {}).get(_active_hz)
            if _layer_data and isinstance(_layer_data, dict):
                _delta = _layer_data.get("delta", {})
                _horizon_label = _active_hz.upper()
                _parts = [f"=== ACTIVE SOLUTION LAYER ({_horizon_label}) ==="]
                _added = _delta.get("added_nodes", [])
                _updated = _delta.get("updated_nodes", [])
                _removed = _delta.get("removed_node_ids", [])
                if _added:
                    _labels = [n.get("label", n.get("id", "?")) for n in _added]
                    _parts.append(f"Added {len(_added)} intervention node(s): {'',
                        ''.join(_labels)}")
                if _updated:
                    _labels = [n.get("label", n.get("id", "?")) for n in _updated]
                    _parts.append(f"Modified {len(_updated)} existing node(s): {'',
                        ''.join(_labels)}")
                if _removed:
                    _parts.append(f"Removed {len(_removed)} node(s): {'',
                        ''.join(_removed)}")
                _parts.append(
                    "\nNote: This solution is a separate snapshot layer. "
                    "The base diagnostic graph has NOT been permanently modified.
                    "
                    "Continue your analysis on the base graph normally."
                )
            ctx.active_solution_summary = "\n".join(_parts)

    # — Paso 4.2: Fetch real graph from GraphEngine for both agents —————
    # graph_json → serialized SessionGraph (nodes + edges + IDs) for the Analyst

```

```

# graph_summary_text → plain NL description (no IDs) for the Interviewer
import json as _json_mod
_sid = str(state.get("session_id", ""))
graph_json = "{}"
graph_summary_text: str | None = None
try:
    from app.graph import get_graph_engine
    _engine = get_graph_engine()
    _session_graph = await _engine.get_current_graph(
        _sid,
        active_framework_id=state.get("active_framework_id"),
    )
    graph_json = _json_mod.dumps(_session_graph.model_dump())
    graph_summary_text = await _engine.get_graph_summary_text(_sid) or None
except Exception as _ge_exc:
    logger.warning(
        "GraphEngine graph fetch failed for session %s (using empty graph):
%s",
        _sid, _ge_exc,
    )

# Prepare Context messages
int_msgs = await context_manager.build_interviewer_context(
    ctx=ctx,
    graph_summary_text=graph_summary_text,
    kb_fragments=None
)

ana_msgs = []
if validated_turn:
    ana_msgs = await context_manager.build_analyst_context(
        ctx=ctx,
        validated_turn=validated_turn,
        graph_json=graph_json,
        kb_fragments=None
    )

from app.core.interview_controller import ObjectiveTracker
tracker = ObjectiveTracker(provider)

stream_cb = state.get("__stream_callback")

# Run agents in parallel
interviewer_coro = interviewer.generate_next_question(int_msgs,
stream_cb=stream_cb)

async def safe_interviewer() -> str:
    try:
        return await asyncio.wait_for(interviewer_coro, timeout=90)
    except asyncio.TimeoutError:

```

```

        logger.error("InterviewerAgent timed out for session %s",
state.get("session_id"))
        return "Could you share one concrete example of when this issue
happens?"
    except Exception as exc:
        logger.error("InterviewerAgent failed for session %s: %s",
state.get("session_id"), exc)
        return "Could you tell me more about this issue with one specific
recent example?"

    async def safe_analyst() -> dict[str, Any] | None:
        if not ana_msgs:
            return None
        try:
            output = await asyncio.wait_for(analyst.analyze_turn(ana_msgs),
timeout=90)
            return output.model_dump()
        except asyncio.TimeoutError:
            logger.error("AnalystAgent timed out for session %s",
state.get("session_id"))
            return None
        except Exception as exc:
            logger.error("AnalystAgent failed: %s", exc)
            return None

    async def safe_tracker() -> dict[str, float]:
        try:
            return await asyncio.wait_for(tracker_coro, timeout=45)
        except asyncio.TimeoutError:
            logger.error("ObjectiveTracker timed out for session %s",
state.get("session_id"))
            return {}
        except Exception as exc:
            logger.error("ObjectiveTracker failed for session %s: %s",
state.get("session_id"), exc)
            return {}

    analyst_coro = safe_analyst()
    tracker_coro = tracker.evaluate_transcript(transcript_objs)

    import time
    _gather_started = time.perf_counter()
    question_text, analyst_out_dict, imo_scores = await asyncio.gather(
        safe_interviewer(), analyst_coro, safe_tracker()
    )
    logger.debug(
        "Interviewing gather complete for session %s in %.2fs",
state.get("session_id"),
time.perf_counter() - _gather_started,
    )

```

```

# — C3: Monotonic merge - IMO scores never decrease once gained —————
existing_scores: dict[str, float] = state.get("imo_scores") or {}
merged_scores: dict[str, float] = {
    key: max(existing_scores.get(key, 0.0), imo_scores.get(key, 0.0))
    for key in set(existing_scores) | set(imo_scores)
}
state["imo_scores"] = merged_scores

if merged_scores and all(score >= 0.7 for score in merged_scores.values()) and
len(merged_scores) >= IMO_COUNT:
    # exploration_complete: canvas can start building, but chat continues.
    # The status is persisted so the frontend knows to show the canvas.
    state["status"] = SessionStatus.exploration_complete.value
    logger.info("Session %s - exploration_complete (all IMOs >= 0.7)",
state.get("session_id"))

# Paso 3.5: Run DomainClassifier on the full transcript
from app.core.domain_classifier import DomainClassifier
classifier = DomainClassifier(provider)
classifications = await classifier.classify(transcript_objs)
state["domain_chips"] = [c.domain for c in classifications]

# — Paso 4.3: Initial framework selection —————
# Only select once - skip if a framework has already been chosen.
if not state.get("active_framework_id"):
    try:
        from app.kb.framework_store import get_framework_store
        _fw_store = get_framework_store()

        # Build transcript excerpt (last 10 user turns)
        _user_turns = [
            m["content"] if isinstance(m, dict) else m.content
            for m in state.get("transcript", [])
            if (m.get("role") if isinstance(m, dict) else m.role) ==
"user"

        ][-10:]
        _excerpt = "\n".join(_user_turns)

        # graph_summary_text was computed above for the interviewer
context

        _fw_id = await analyst.select_framework(
            domain_chips=state.get("domain_chips", []),
            graph_summary=graph_summary_text or "",
            transcript_excerpt=_excerpt,
            store=_fw_store,
        )

        _fw_info = _fw_store.get(_fw_id)
        state["active_framework_id"] = _fw_id

```

```

state["active_framework"] = _fw_info.model_dump()
state["framework_selected_by"] = "analyst"
logger.info(
    "Session %s - framework selected: %s (%s)",
    state.get("session_id"), _fw_id, _fw_info.display_name,
)

# Snapshot the graph at framework selection time (non-fatal)
try:
    from app.graph import get_graph_engine
    from app.models.session import SnapshotTrigger
    _snap_engine = get_graph_engine()
    await _snap_engine.snapshot(
        session_id=str(state.get("session_id", "")),
        trigger=SnapshotTrigger.framework_selected,
    )
except Exception as _snap_exc:
    logger.warning(
        "Snapshot after framework selection failed (non-fatal):
%s", _snap_exc
    )

except Exception as _fw_exc:
    logger.error(
        "Framework selection failed for session %s (non-fatal): %s",
        state.get("session_id"), _fw_exc,
    )

# — Generate full initial graph (ONE-SHOT - guarded by graph_initialized)

# Only runs the first time all IMO's reach 0.7. Subsequent turns
# (including post-proposals_ready chat) skip this block entirely.
if not state.get("graph_initialized", False):
    try:
        # Reconstruct FrameworkInfo for context building
        _initial_fw = None
        _initial_fw_raw = state.get("active_framework")
        if _initial_fw_raw:
            try:
                _initial_fw = FrameworkInfo(
                    **(_initial_fw_raw if isinstance(_initial_fw_raw,
dict)
                    else _initial_fw_raw.model_dump())
                )
            except Exception:
                pass

        # Build context with FULL transcript (exception to D-IA-02)
        _initial_ctx = SessionContext(
            session_id=state["session_id"],

```

```

        status=SessionStatus(state["status"]),
        transcript=transcript_objs,
        validated_metrics=validated_metrics,
        imo_scores=state.get("imo_scores", {}),
        graph_summary=None,
        analyst_output=None,
        active_framework=_initial_fw,
    )

    _initial_msgs = await context_manager.build_initial_graph_context(
        ctx=_initial_ctx,
        domain_chips=state.get("domain_chips", []),
    )

    _initial_output = await
analyst.generate_initial_graph(_initial_msgs)

    # Override the per-turn delta with the full graph
    analyst_out_dict = _initial_output.model_dump()

    # Persist full graph to Neo4j
    try:
        from app.graph import get_graph_engine
        _ig_engine = get_graph_engine()
        await _ig_engine.add_analyst_output(
            session_id=str(state.get("session_id", "")),
            output=_initial_output,
        )
        # Mark graph as initialized - subsequent turns will persist
        # incremental deltas normally
        state["graph_initialized"] = True
        state["graph_revision"] = int(state.get("graph_revision", 0)
or 0) + 1

        # Initial graph is always structural
        state["structural_revision"] =
int(state.get("structural_revision", 0) or 0) + 1
        logger.info(
            "Initial graph persisted to Neo4j for session %s: %d
nodes, %d edges (rev=%d, struct_rev=%d)",
            state.get("session_id"),
            len(_initial_output.graph_delta.added_nodes),
            len(_initial_output.graph_delta.added_edges),
            state["graph_revision"],
            state["structural_revision"],
        )

        # — Override question_text with a graph-ready announcement

        # Replace the interviewer's plain question with a message
that:

```

```

# (a) announces the diagram was generated
# (b) invites the user to add more detail
# (c) includes 1-2 context-specific follow-up questions
try:
    _node_count = len(_initial_output.graph_delta.added_nodes)
    _recent_user = [
        (m.content if hasattr(m, "content") else
m.get("content", ""))
        for m in transcript_objs
        if (m.role if hasattr(m, "role") else m.get("role"))
in ("user", MessageRole.user)
    ][-4:]
    _recent_excerpt = "\n".join(_recent_user)

    _lang_hint = (
        "Detect the language used in the conversation excerpt
and reply in that same language."
    )
    _announce_prompt = (
        f"You are a business process consultant who just
mapped an initial process diagram "
        f"containing {_node_count} elements based on the
client's information.\n\n"
        f"Recent conversation excerpt:\n{_recent_excerpt}\n\n"
        f"Write a short chat message (2-4 sentences) that:\n"
        f"1. States directly (NO greeting, NO 'Hola', NO
'Hello') that an initial version "
        f"  of the process diagram has been generated on the
canvas.\n"
        f"2. Invites the user to keep sharing information to
make it more detailed and accurate.\n"
        f"3. Ends with 1-2 specific follow-up questions based
on the conversation context "
        f"  to gather information about process gaps,
volumes, timings, or responsible teams.\n\n"
        f"{_lang_hint}\n"
        f"CRITICAL: Do NOT start with any greeting (no 'Hola',
no 'Hello', no 'Hi', no '¡Hola!').\n"
        f"Start directly with the announcement. Write ONLY the
message itself, no titles or headers."
    )
    _announce_msg = await provider.generate(
        [ProviderMessage(role="user",
content=_announce_prompt)],
        temperature=0.5,
        max_tokens=300,
    )
    question_text = _announce_msg.strip()
    logger.info(
        "Initial graph announcement generated for session %s",

```

```

        state.get("session_id"),
    )
    except Exception as _ann_exc:
        logger.warning("Graph announcement generation failed
(non-fatal): %s", _ann_exc)
        # Fallback to a static announcement if LLM fails
        question_text = (
            "An initial version of the process diagram has been
generated on the canvas. "
            "You can continue sharing information to make it more
detailed. "
            "Is there a specific team or area whose workflow you'd
like us to dive deeper into?"
        )
    except Exception as _neo4j_exc:
        logger.error(
            "Initial graph Neo4j persistence failed (non-fatal): %s",
            _neo4j_exc
        )

    except Exception as _ig_exc:
        logger.error(
            "Initial graph generation failed for session %s (non-fatal, "
            "per-turn delta will be used instead). error_type=%s
error=%s",

            state.get("session_id"),
            type(_ig_exc).__name__,
            _ig_exc,
        )
        _raw_output = getattr(_ig_exc, "raw_output", None)
        if isinstance(_raw_output, str) and _raw_output.strip():
            logger.error(
                "Initial graph agent raw output (session=%s,
first_8k_chars):\n---\n%s\n---",
                state.get("session_id"),
                _raw_output[:8000],
            )

        # Generate suggestion chips for the question
        try:
            import json as _json
            chips_prompt = (
                f"Given the following diagnostic interview
question:\n\"{question_text}\"\\n\\n"
                "Generate exactly 3 short answer options (max 12 words each) that the
user could "
                "choose as quick responses. The options should cover different angles
to the question. "
                "Write them in the same language as the question. "
                "Return ONLY a JSON array of strings, no other text. "

```

```

        "Example: [\"Option 1\", \"Option 2\", \"Option 3\"]"
    )
    chips_raw = await provider.generate(
        [ProviderMessage(role="user", content=chips_prompt)],
        temperature=0.5,
        max_tokens=512,
    )
    # Handle case where generate returns an async generator instead of string
    if not isinstance(chips_raw, str):
        logger.warning("Suggestion chips: generate returned non-string type
%s, skipping", type(chips_raw).__name__)
        state["__suggestion_chips"] = []
    else:
        chips_text = chips_raw.strip()
        start_idx = chips_text.find("[")
        end_idx = chips_text.rfind("]")
        if start_idx != -1 and end_idx != -1:
            chips_text = chips_text[start_idx:end_idx + 1]
            state["__suggestion_chips"] = _json.loads(chips_text)
    except Exception as exc:
        logger.warning(
            "Failed to generate suggestion chips: %s\nRaw output:\n---\n%s\n---",
            exc, chips_raw if 'chips_raw' in dir() else '(no output)',
        )
        state["__suggestion_chips"] = []

    # Validate how many turns have passed
    user_turn_count = sum(1 for m in transcript_objs if m.role == "user" or m.role
== MessageRole.user)

    # — C5: Auto-title on first user turn —————
    # If this is the very first user message, generate a short session title.
    if user_turn_count == 1:
        try:
            first_msg = next(
                (m.content for m in transcript_objs if m.role == "user" or m.role
== MessageRole.user),
                "",
            )
            title_prompt = ProviderMessage(
                role="user",
                content=(
                    f"Generate a very short session title (max 6 words) that
captures the main "
                    f"business problem described in this text:
\"{first_msg[:400]}\"\\n"
                    "Reply with ONLY the title itself, no quotes, no punctuation
at the end."
                ),
            )

```

```

        raw_title = await provider.generate(
            [title_prompt], temperature=0.3, max_tokens=30
        )
        state["__session_title"] = raw_title.strip().strip(' ').strip()[:120]
        logger.info("Auto-title generated for session %s: %s",
            state.get("session_id"), state["__session_title"])
    except Exception as title_exc:
        logger.warning("Auto-title generation failed: %s", title_exc)

# Update state
if analyst_out_dict:
    analyst_out_dict["imo_coverage_update"] = merged_scores

# — Graph delta gating —————
# Only send graph_delta to the frontend (and persist to Neo4j) when
# the initial graph has already been generated. During the interviewing
# phase, the Analyst still runs to track IMOs and produce
exploration_gaps,
# but its graph_delta is suppressed – the canvas stays empty until
# exploration_complete fires generate_initial_graph().
graph_is_initialized = state.get("graph_initialized", False)

if not graph_is_initialized:
    # Pre-initialization: suppress graph_delta so canvas stays empty
    # and skip Neo4j persistence.
    empty_delta = {
        "added_nodes": [], "updated_nodes": [], "removed_node_ids": [],
        "added_edges": [], "updated_edges": [], "removed_edge_ids": []
    }
    analyst_out_dict["graph_delta"] = empty_delta
    logger.debug(
        "Session %s: graph_delta suppressed (graph not yet initialized)",
        state.get("session_id"),
    )

state["analyst_output"] = analyst_out_dict

# — Paso 4.1: Persist graph delta to Neo4j via GraphEngine —————
# Only persist incremental deltas AFTER the initial graph has been
# generated (graph_initialized == True). The initial graph itself is
# already persisted directly in the exploration_complete block above.
if graph_is_initialized:
    gd = analyst_out_dict.get("graph_delta", {})
    has_content = (
        gd.get("added_nodes") or gd.get("updated_nodes") or
        gd.get("added_edges") or gd.get("updated_edges") or
        gd.get("removed_node_ids") or gd.get("removed_edge_ids")
    )
    if has_content:
        try:

```

```

from app.models.session import AnalystOutput as _AnalystOutput
from app.graph import get_graph_engine
_analyst_output_obj = _AnalystOutput(**analyst_out_dict)
_engine = get_graph_engine()
await _engine.add_analyst_output(
    session_id=str(state.get("session_id", "")),
    output=_analyst_output_obj,
)
state["graph_revision"] = int(state.get("graph_revision", 0)
or 0) + 1

# Check if this delta is structural (node/edge add/remove)
_is_structural = bool(
    gd.get("added_nodes") or gd.get("removed_node_ids")
    or gd.get("added_edges") or gd.get("removed_edge_ids")
)
if _is_structural:
    state["structural_revision"] =
int(state.get("structural_revision", 0) or 0) + 1
    # Mark solution layers and proposals as stale
    _sl = state.get("solution_layers")
    if _sl and isinstance(_sl, dict):
        for _lk, _lv in _sl.get("layers", {}).items():
            if isinstance(_lv, dict):
                _lv["is_stale"] = True
    logger.debug(
        "GraphEngine persisted delta for session %s: +%d nodes +%d
edges (rev=%d, struct=%s)",
        state.get("session_id"),
        len(gd.get("added_nodes", [])),
        len(gd.get("added_edges", [])),
        state["graph_revision"],
        _is_structural,
    )
# — Graph history checkpoint after each AI turn —————
try:
    from app.core.graph_history import graph_history_manager
    from app.db.base import SessionLocal
    _added = len(gd.get("added_nodes", []))
    _upd    = len(gd.get("updated_nodes", []))
    _desc_parts = []
    if _added: _desc_parts.append(f"+{_added} nodes")
    if _upd:   _desc_parts.append(f"{_upd} updated")
    if not _desc_parts: _desc_parts.append("diagnostic
changes")

    _ckpt_desc = f"Diagnostic turn - {' '.join(_desc_parts)}"
    _db = SessionLocal()
    try:
        await graph_history_manager.create_checkpoint(
            session_id=str(state.get("session_id", "")),
            db=_db,

```

```

        engine=_engine,
        description=_ckpt_desc,
    )
    finally:
        _db.close()
except Exception as _ckpt_exc:
    logger.debug("analyst turn: checkpoint skipped: %s",
_ckpt_exc)

except Exception as graph_exc:
    logger.error(
        "GraphEngine.add_analyst_output failed for session %s:
%s",

        state.get("session_id"), graph_exc,
    )

state["transcript"].append({"role": "assistant", "content": question_text})
return state

async def _node_exploration_complete(state: dict[str, Any]) -> dict[str, Any]:
    """Exploration phase finished – all 6 IMOs covered above threshold.

    The conversation continues in this state; the canvas starts building in
    parallel.
    Stub: in Paso 4.3 this will trigger framework selection. Until then it simply
    stays in exploration_complete so stream_turn can keep routing to
    _node_interviewing.
    """
    logger.debug("Node: exploration_complete for session %s",
state.get("session_id"))
    # Do NOT auto-advance to audit_pending – that blocks the chat.
    # The real transition is triggered manually / in Paso 5.1 (CausalAuditor).
    return state

async def _node_audit_pending(state: dict[str, Any]) -> dict[str, Any]:
    """CausalAuditor running – checks graph for structural issues.

    Stub: in Paso 5.1 this will call CausalAuditor and block on findings.
    If no blocking findings, transitions to audit_passed automatically.
    """
    logger.debug("Node: audit_pending for session %s", state.get("session_id"))
    run_audit_cb = state.get("__run_audit_callback")
    if callable(run_audit_cb):
        return await run_audit_cb(state)
    logger.warning(
        "audit_pending node executed without __run_audit_callback for session %s",
state.get("session_id"),

```

```
)  
return state
```

```
async def _node_audit_passed(state: dict[str, Any]) -> dict[str, Any]:  
    """Graph validated – gap analysis phase.
```

```
  
    On the FIRST entry (no target_metrics yet), generates the opening question  
    and emits it as a gap_analysis_question WS event.  
    On subsequent entries (user answered a metric), this node is not re-invoked  
    directly – the routing goes through _node_interviewing which delegates to  
    GapAnalysisController when status == audit_passed.  
    """
```

```
    logger.debug("Node: audit_passed for session %s", state.get("session_id"))  
    run_gap_cb = state.get("__run_gap_analysis_callback")  
    if callable(run_gap_cb):  
        return await run_gap_cb(state)  
    return state
```

```
async def _node_proposals_ready(state: dict[str, Any]) -> dict[str, Any]:  
    """Terminal state – proposals generated and ready for presentation."""  
    logger.debug("Node: proposals_ready for session %s", state.get("session_id"))  
    return state
```

```
# — Router: decides the next node from session status —————
```

```
def _route_from_status(state: dict[str, Any]) -> str:  
    """LangGraph conditional edge – routes to the next node based on status."""  
    status = state.get("status", SessionStatus.idle)  
    routing: dict[str, str] = {  
        SessionStatus.idle: "idle",  
        SessionStatus.interviewing: "interviewing",  
        SessionStatus.exploration_complete: "exploration_complete",  
        SessionStatus.audit_pending: "audit_pending",  
        SessionStatus.proposal_inputs_pending: "audit_pending",  
        SessionStatus.audit_passed: "audit_passed",  
        SessionStatus.proposals_ready: "proposals_ready",  
    }  
    return routing.get(status, END) # type: ignore[return-value]
```

```
# — LangGraph graph compilation (module-level singleton) —————
```

```
def _build_graph() -> Any:  
    """Build and compile the LangGraph state machine for a session.
```

The graph is stateless – state is passed in via the input dict on each invocation. This allows reuse of the same compiled graph across all sessions.

```
"""
from langgraph.graph import START
builder: StateGraph = StateGraph(dict)

# Register nodes
builder.add_node("idle", _node_idle)
builder.add_node("interviewing", _node_interviewing)
builder.add_node("exploration_complete", _node_exploration_complete)
builder.add_node("audit_pending", _node_audit_pending)
builder.add_node("audit_passed", _node_audit_passed)
builder.add_node("proposals_ready", _node_proposals_ready)

# Entry point dispatcher
builder.add_conditional_edges(START, _route_from_status)

# All nodes route to END after processing (turn-by-turn execution model).
# The caller decides when to re-enter the graph based on the persisted state.
for node in [
    "idle",
    "interviewing",
    "audit_passed",
    "proposals_ready",
]:
    builder.add_edge(node, END)

# Nodes that auto-transition to the next status
builder.add_edge("exploration_complete", END)
builder.add_edge("audit_pending", END)

return builder.compile()
```

```
_session_graph = _build_graph()
```

```
# — SessionController —————
```

```
class SessionController:
```

```
    """Orchestrates the session lifecycle.
```

This is the ONLY class allowed to transition session states (D-IA-01).
All state changes go through here and are persisted to PostgreSQL.

Paso 3.1 scope: create_session, process_turn (stub), get_session.
Agent calls (Interviewer/Analyst) are wired in Paso 3.2.
MetricValidator is wired in Paso 3.4.
DomainClassifier is wired in Paso 3.5.

```

"""

def __init__(self, db: Session) -> None:
    """
    Args:
        db: SQLAlchemy session for PostgreSQL persistence. Injected
            by FastAPI via the get_db dependency.
    """
    self._db = db

# — Public API —————

def create_session(
    self,
    workspace_id: str,
    user_id: str,
    title: str | None = None,
) -> SessionState:
    """Create a new diagnostic session in `idle` state.

    Persists to PostgreSQL and returns the initial SessionState.
    The session starts in `idle` – it only moves to `interviewing` after
    the first user message is confirmed (process_turn).

    Args:
        workspace_id: UUID of the workspace creating the session.
        user_id: UUID of the user who owns this session.
        title: Optional human-readable title (auto-generated if omitted).

    Returns:
        SessionState with the new session_id and status=idle.
    """
    session_id = str(uuid.uuid4())

    db_session = DBSession(
        id=uuid.UUID(session_id),
        workspace_id=uuid.UUID(workspace_id),
        created_by=uuid.UUID(user_id),
        title=title,
        status=SessionStatus.idle,
        transcript=[],
        imo_scores={},
        validated_metrics=[],
        audit_findings=[],
    )
    self._db.add(db_session)
    self._db.commit()
    self._db.refresh(db_session)

```

```
logger.info("Session created: %s (workspace=%s)", session_id,
workspace_id)
```

```
return SessionState(
    session_id=session_id,
    status=SessionStatus.idle,
)
```

```
async def process_turn(
    self,
    session_id: str,
    user_message: str,
) -> SessionState:
```

```
    """Process one turn of the interview – the core session loop.
```

```
    Current (Paso 3.1): stub that transitions idle→interviewing and
    echoes the message back. The Interviewer/Analyst agents are wired
    in Paso 3.2.
```

```
    Args:
```

```
        session_id: The session to advance.
```

```
        user_message: The raw text submitted by the user.
```

```
    Returns:
```

```
        The updated SessionState after the turn.
```

```
    Raises:
```

```
        ValueError: If session_id is not found.
```

```
        InvalidTransitionError: If the current status does not allow
        the `interviewing` transition.
```

```
    """
```

```
    state = self._load_state(session_id)
```

```
    # First user message: transition idle → interviewing
```

```
    if state.status == SessionStatus.idle:
```

```
        state = self._transition(state, SessionStatus.interviewing)
```

```
    # Map workspace to obtain the correct provider
```

```
    db_sess = self._db.query(DBSession).filter(DBSession.id ==
uuid.UUID(session_id)).first()
```

```
    workspace = db_sess.workspace
```

```
    from app.providers.factory import get_provider
```

```
    provider = get_provider(workspace)
```

```
    # 1. MetricValidator.validate(user_message, context) (Paso 3.4)
```

```
    # 2. Append to transcript
```

```
    state.transcript.append(
```

```
        Message(role=MessageRole.user, content=user_message)
```

```
    )
```

```

if state.status == SessionStatus.proposal_inputs_pending:

    async def _noop_stream(_: str) -> None:
        return None

    final_state = await self._run_proposal_readiness_turn(
        state=state,
        user_message=user_message,
        provider=provider,
        stream_cb=_noop_stream,
    )
    self._persist_state(session_id, final_state)
    return final_state

# Run the LangGraph node for the current status
raw_state = _state_to_dict(state)
# Inject transient dependencies required by the graph nodes
raw_state["__provider"] = provider
raw_state["__run_audit_callback"] = self._run_audit_graph_node

# We must use `ainvoke` because our LangGraph nodes are async
result = await _session_graph.ainvoke(raw_state)

# Cleanup transient before dict mapping
result.pop("__provider", None)
result.pop("__run_audit_callback", None)
final_state = _dict_to_state(result)
self._persist_state(session_id, final_state)
return final_state

async def stream_turn(
    self,
    session_id: str,
    user_message: str,
    stream_cb: Callable[[str], Awaitable[None]]
) -> tuple[SessionState, list[str], str | None]:
    """Process one turn using WebSocket streaming for the interviewer agent.

    Returns:
        Tuple of (final SessionState, suggestion_chips list, session_title |
None).
    """
    state = self._load_state(session_id)

# — C2: exploration_complete keeps the interview running —————
# Once all IMO's hit 70% the status becomes exploration_complete so the
# canvas can start building – but the user can keep chatting.
if state.status not in _CHAT_ACTIVE_STATUSES:
    raise InvalidTransitionError(state.status, SessionStatus.interviewing)

```

```

# Remember the true pre-turn status before we route through interviewing
_original_status = state.status

if state.status == SessionStatus.idle:
    state = self._transition(state, SessionStatus.interviewing)
elif state.status == SessionStatus.exploration_complete:
    # exploration_complete keeps chat active and routes to interviewing.
    state = state.model_copy(update={"status":
SessionStatus.interviewing})
elif state.status == SessionStatus.proposals_ready:
    # proposals_ready: route through interviewing so both agents run,
    # but remember we came from proposals_ready so we can restore it.
    state = state.model_copy(update={"status":
SessionStatus.interviewing})
    # proposal_inputs_pending: keep persisted status for readiness follow-up

# Provider logic
from app.db.models import Session as DBSession
db_sess = self._db.query(DBSession).filter(DBSession.id ==
uuid.UUID(session_id)).first()
workspace = db_sess.workspace
from app.providers.factory import get_provider
provider = get_provider(workspace)

state.transcript.append(
    Message(role=MessageRole.user, content=user_message)
)

# — C7: audit_passed → direct proposal generation —————
# When the session is stuck at audit_passed (e.g. provider was unavailable
# during the audit, or this is an older session), any user message
triggers
# immediate proposal generation. The gap-analysis Q&A round is bypassed.
if state.status == SessionStatus.audit_passed:
    logger.info(
        "Session %s at audit_passed - generating proposals on user
message.",
        session_id,
    )
    final_state = await self._generate_proposals(state, provider)
    self._persist_state(session_id, final_state)
    return final_state, [], None

if state.status == SessionStatus.proposal_inputs_pending:
    final_state = await self._run_proposal_readiness_turn(
        state=state,
        user_message=user_message,
        provider=provider,
        stream_cb=stream_cb,
    )

```

```

        self._persist_state(session_id, final_state)
        return final_state, [], None

    raw_state = _state_to_dict(state)
    raw_state["__provider"] = provider
    raw_state["__stream_callback"] = stream_cb
    raw_state["__run_audit_callback"] = self._run_audit_graph_node

    result = await _session_graph.ainvoke(raw_state)

    # Extract transient data before cleanup
    suggestion_chips = result.pop("__suggestion_chips", [])
    session_title: str | None = result.pop("__session_title", None)
    result.pop("__provider", None)
    result.pop("__stream_callback", None)
    result.pop("__run_audit_callback", None)
    final_state = _dict_to_state(result)

    # — Status regression guard —————
    # If the session was at proposals_ready before this turn, the Analyst
    # node re-evaluates IMO scores and may reset status to
    exploration_complete.
    # We must restore proposals_ready so the frontend doesn't show the
    # "Generar propuesta" button again and re-open the overlay.
    if _original_status == SessionStatus.proposals_ready and
    final_state.status in {
        SessionStatus.exploration_complete, SessionStatus.interviewing
    }:
        final_state = final_state.model_copy(update={"status":
SessionStatus.proposals_ready})
        logger.debug(
            "stream_turn: status restored to proposals_ready after regression
(session=%s)",
            session_id,
        )

    self._persist_state(session_id, final_state)

    # — C5: Persist auto-generated title to DB —————
    if session_title:
        db_sess = self._db.query(DBSession).filter(
            DBSession.id == uuid.UUID(session_id)
        ).first()
        if db_sess and not db_sess.title:
            db_sess.title = session_title
            self._db.commit()
            logger.info("Session %s titled: '%s'", session_id, session_title)

    return final_state, suggestion_chips, session_title

```

```

async def run_audit(
    self,
    session_id: str,
    recheck_node_ids: list[str] | None = None,
) -> SessionState:
    """Run causal audit and persist findings/status transitions (REST
path)."""
    return await self.run_audit_with_progress(
        session_id=session_id,
        recheck_node_ids=recheck_node_ids,
    )

async def run_audit_with_progress(
    self,
    session_id: str,
    recheck_node_ids: list[str] | None = None,
    progress_cb: "Callable[[dict], Awaitable[None]] | None" = None,
) -> SessionState:
    """Run causal audit, emitting granular WS events via progress_cb.

    Args:
        session_id: Session to audit.
        recheck_node_ids: Optional subset of node IDs for incremental
re-audit.
        progress_cb: Async callable that receives progress dicts to forward to
the WebSocket client. If None, runs silently (REST path).

    The progress events emitted are:
        - audit_started          → pipeline has begun
        - audit_progress          → step-level granularity
        - audit_complete          → final result with findings / readiness
prompt
    """

    async def _noop(_: dict) -> None:
        return None

    emit = progress_cb or _noop

    state = self._load_state(session_id)

    # Fetch provider for auto-proposal generation if audit passes cleanly
    provider = None
    try:
        db_sess = self._db.query(DBSession).filter(DBSession.id ==
uuid.UUID(session_id)).first()
        if db_sess:
            from app.providers.factory import get_provider
            provider = get_provider(db_sess.workspace)
    except Exception as _prov_exc:

```

```

        logger.warning("run_audit: could not fetch provider (non-fatal): %s",
            _prov_exc)

        await emit({
            "type": "audit_started",
            "message": "Iniciando validación del grafo causal...",
        })

        audited = await self._execute_audit(
            state=state,
            recheck_node_ids=recheck_node_ids,
            provider=provider,
            progress_cb=emit,
        )
        self._persist_state(session_id, audited)

        # Emit terminal event so the frontend knows what happened
        findings = [f.model_dump(mode="json") for f in audited.audit_findings]
        blocking = [f for f in audited.audit_findings if f.is_blocking]
        final_status = (
            audited.status.value if hasattr(audited.status, "value") else
str(audited.status)
        )
        logger.debug(
            "run_audit_with_progress(%s): emitting audit_complete - status=%s, "
            "total_findings=%d, blocking=%d, first_finding=%s",
            session_id, final_status, len(findings), len(blocking),
            findings[0] if findings else None,
        )

        if blocking:
            await emit({
                "type": "audit_complete",
                "result": "blocked",
                "status": final_status,
                "findings": findings,
                "blocking_count": len(blocking),
                "warning_count": len(findings) - len(blocking),
                "message": (
                    f"{len(blocking)} problema{'s' if len(blocking) > 1 else ''}
en el grafo "
                    f"requiere{'n' if len(blocking) > 1 else ''} atención antes de
continuar."
                ),
            })
        elif final_status == "proposal_inputs_pending":
            readiness_prompt = audited.metadata.get("__readiness_last_prompt", "")
            await emit({
                "type": "audit_complete",
                "result": "inputs_needed",

```

```

        "status": final_status,
        "findings": findings,
        "blocking_count": 0,
        "warning_count": len(findings),
        "message": str(readiness_prompt),
    })
else:
    await emit({
        "type": "audit_complete",
        "result": "passed",
        "status": final_status,
        "findings": findings,
        "blocking_count": 0,
        "warning_count": len(findings),
        "message": "Graph validated ✓ Starting gap analysis...",
    })

    return audited

    async def start_edit_mode(self, session_id: str) -> tuple[str, "dict[str,
Any]"]]:
        """Enable edit mode and snapshot the current graph state."""
        from app.graph import get_graph_engine
        from app.models.session import SnapshotTrigger

        state = self._load_state(session_id)
        if state.status not in {
            SessionStatus.exploration_complete,
            SessionStatus.audit_pending,
            SessionStatus.audit_passed,
            SessionStatus.proposal_inputs_pending,
        }:
            raise InvalidTransitionError(state.status,
SessionStatus.exploration_complete)

        engine = get_graph_engine()
        snapshot_id = await engine.snapshot(session_id=session_id,
trigger=SnapshotTrigger.edit_started)
        graph = await engine.get_current_graph(
            session_id=session_id,
            active_framework_id=state.active_framework_id,
        )
        metadata = dict(state.metadata or {})
        metadata.update(
            {
                "edit_mode_active": True,
                "edit_base_snapshot_id": snapshot_id,
                "edit_started_at": datetime.now(timezone.utc).isoformat(),
            }
        )

```

```

        updated_state = state.model_copy(update={"metadata": metadata})
        self._persist_state(session_id, updated_state)
        return snapshot_id, graph.model_dump(mode="json")

    async def commit_edit_mode(
        self,
        session_id: str,
        graph_changes: GraphDelta,
        user_explanation: str,
        base_snapshot_id: str | None = None,
    ) -> tuple[SessionState, GraphDelta]:
        """Commit manual graph changes, run analyst impact analysis, and
        snapshot."""
        from app.agents.analyst import AnalystAgent
        from app.graph import get_graph_engine
        from app.models.session import AnalystOutput, SnapshotTrigger
        from app.providers.factory import get_provider

        state = self._load_state(session_id)
        metadata = dict(state.metadata or {})

        engine = get_graph_engine()
        snapshot_id = str(base_snapshot_id or "").strip()
        if snapshot_id:
            try:
                pre_graph = await engine.get_snapshot_graph(
                    session_id=session_id,
                    snapshot_id=snapshot_id,
                )
            except Exception:
                pre_graph = await engine.get_current_graph(
                    session_id=session_id,
                    active_framework_id=state.active_framework_id,
                )
        else:
            pre_graph = await engine.get_current_graph(
                session_id=session_id,
                active_framework_id=state.active_framework_id,
            )

        await engine.apply_graph_delta(session_id=session_id, delta=graph_changes)
        post_graph = await engine.get_current_graph(
            session_id=session_id,
            active_framework_id=state.active_framework_id,
        )
        structural_diff = self._compute_structural_diff(pre_graph=pre_graph,
        post_graph=post_graph)
        dependent_node_ids = self._collect_direct_dependents(
            graph=post_graph,
            changed_node_ids=self._changed_node_ids_from_delta(structural_diff),

```

```

    )

    db_sess = self._db.query(DBSession).filter(DBSession.id ==
uuid.UUID(session_id)).first()
    if db_sess is None:
        raise ValueError(f"Session not found: {session_id}")
    provider = get_provider(db_sess.workspace)
    analyst = AnalystAgent(provider)
    analyst_output = await analyst.analyze_edit_commit(
        graph_changes=structural_diff,
        user_explanation=user_explanation,
        post_graph=post_graph,
        dependent_node_ids=dependent_node_ids,
        active_framework=state.active_framework,
        validated_metrics=state.validated_metrics,
    )
    confirmed_delta = await engine.add_analyst_output(
        session_id=session_id,
        output=analyst_output,
    )
    final_analyst_output = AnalystOutput(
        graph_delta=confirmed_delta,
        exploration_gaps=analyst_output.exploration_gaps,
        emerging_hypotheses=analyst_output.emerging_hypotheses,
        imo_coverage_update=analyst_output.imo_coverage_update,
        kb_match=analyst_output.kb_match,
        framework_context=analyst_output.framework_context,
    )

    metadata["last_edit_commit_at"] = datetime.now(timezone.utc).isoformat()
    metadata["last_edit_structural_diff"] =
structural_diff.model_dump(mode="json")

    revision = int(state.graph_revision or 0)
    if self._delta_has_content(graph_changes):
        revision += 1
    if self._delta_has_content(confirmed_delta):
        revision += 1

    updated_state = state.model_copy(
        update={
            "metadata": metadata,
            "analyst_output": final_analyst_output,
            "graph_revision": revision,
        }
    )
    self._persist_state(session_id, updated_state)

    await engine.snapshot(session_id=session_id,
trigger=SnapshotTrigger.edit_committed)

```

```

        return updated_state, structural_diff

    async def change_framework(
        self,
        session_id: str,
        framework_id: str,
        ws_send: "Callable[[dict], Awaitable[None]] | None" = None,
    ) -> "GraphRestructureOutput":
        """Switch the active diagnostic framework for a session.

        This is the FRAMEWORK_CHANGED handler (D-IA-01). The Analyst is invoked
        in restructure mode – it reclassifies every node in the current graph
        under the new framework's zones without adding or removing nodes/edges.

        Args:
            session_id: The session to update.
            framework_id: ID of the new framework (must exist in FrameworkStore).
            ws_send: Optional async callable to push WebSocket events back to
                    the connected client. Receives a dict payload.

        Returns:
            The GraphRestructureOutput produced by the Analyst.

        Raises:
            KeyError: If framework_id is not in the FrameworkStore.
            ValueError: If framework_id == active_framework_id (no-op).
            ValueError: If session is not found.
        """
        from app.agents.analyst import AnalystAgent
        from app.graph import get_graph_engine
        from app.kb.framework_store import get_framework_store
        from app.models.session import (
            FrameworkChange,
            FrameworkInfo,
            GraphRestructureOutput,
        )
        from app.providers.factory import get_provider

        state = self._load_state(session_id)

        # Validate framework exists
        fw_store = get_framework_store()
        new_fw = fw_store.get(framework_id) # raises KeyError if not found

        # Reject no-op switches
        if state.active_framework_id == framework_id:
            raise ValueError(
                f"Framework '{framework_id}' is already active for session {session_id}."
            )

```

```

engine = get_graph_engine()
graph_revision = int(state.graph_revision or 0)
cache_by_framework = dict(state.framework_restructure_cache or {})
cache_hit = False
cached_entry = cache_by_framework.get(framework_id)
restructure_output: GraphRestructureOutput

if (
    isinstance(cached_entry, dict)
    and int(cached_entry.get("base_graph_revision", -1)) == graph_revision
    and isinstance(cached_entry.get("restructure_output"), dict)
):
    try:
        restructure_output =
GraphRestructureOutput(**cached_entry["restructure_output"])
        cache_hit = True
    except Exception as cache_exc:
        logger.warning(
            "Framework cache parse failed for session %s framework %s:
%s",

            session_id,
            framework_id,
            cache_exc,
        )
        cache_hit = False

if not cache_hit:
    # Resolve provider from workspace
    db_sess = self._db.query(DBSession).filter(
        DBSession.id == uuid.UUID(session_id)
    ).first()
    if db_sess is None:
        raise ValueError(f"Session not found: {session_id}")
    provider = get_provider(db_sess.workspace)

    # Get full current graph from Neo4j and run Analyst in restructure
mode
current_graph = await engine.get_current_graph(
    session_id,
    active_framework_id=state.active_framework_id,
)
analyst = AnalystAgent(provider)
restructure_output = await analyst.restructure_graph(
    current_graph=current_graph,
    new_framework=new_fw,
    validated_metrics=state.validated_metrics,
)
cache_by_framework[framework_id] = {
    "base_graph_revision": graph_revision,

```

```

        "restructure_output": restructure_output.model_dump(mode="json"),
        "updated_at": datetime.now(timezone.utc).isoformat(),
    }

# Apply to Neo4j (updates zones/types, creates snapshot automatically)
await engine.restructure(session_id, restructure_output)

# Record history entry
change_record = FrameworkChange(
    previous_framework_id=state.active_framework_id or "",
    new_framework_id=framework_id,
    changed_by="user",
)
updated_history = list(state.framework_change_history) + [change_record]

# Update state
state = state.model_copy(
    update={
        "active_framework_id": framework_id,
        "active_framework": new_fw,
        "framework_selected_by": "user",
        "framework_change_history": updated_history,
        "restructure_output": restructure_output,
        "framework_restructure_cache": cache_by_framework,
    }
)
self._persist_state(session_id, state)

logger.info(
    "SessionController.change_framework: session=%s %s-%s cache_hit=%s
rev=%s",
    session_id,
    change_record.previous_framework_id or "(none)",
    framework_id,
    cache_hit,
    graph_revision,
)

restored_positions: dict[str, dict[str, float]] = {}
try:
    graph_after = await engine.get_current_graph(
        session_id,
        active_framework_id=framework_id,
    )
    for node in graph_after.nodes:
        if node.position is None:
            continue
        restored_positions[node.id] = {
            "x": float(node.position.x),
            "y": float(node.position.y),

```

```

    }
    except Exception as restore_exc:
        logger.warning(
            "change_framework: could not load restored positions for session
%s framework %s: %s",
            session_id,
            framework_id,
            restore_exc,
        )

    # Emit WebSocket events if a sender is available
    if ws_send is not None:
        try:
            await ws_send({
                "type": "canvas_restructure",
                "output": restructure_output.model_dump(mode="json"),
                "framework_id": framework_id,
                "cache_hit": cache_hit,
                "graph_revision": graph_revision,
                "node_positions": restored_positions,
            })
        except Exception as ws_exc:
            logger.warning(
                "change_framework: canvas_restructure WS send failed: %s",
                ws_exc
            )
        try:
            await ws_send({
                "type": "chat_message",
                "content": restructure_output.consultant_summary,
            })
        except Exception as ws_exc:
            logger.warning(
                "change_framework: chat_message WS send failed: %s", ws_exc
            )

    return restructure_output

async def _apply_audit_passed_success(
    self,
    state: SessionState,
    engine: Any,
    provider: Any = None,
) -> SessionState:
    """Snapshot, then auto-generate proposals immediately.

```

As soon as the audit passes, we generate H1/H2/H3 proposals without any gap-analysis Q&A round. The proposals are generated using whatever context is already available (validated_metrics, graph summary, framework).

```

Gap analysis (target metric collection) is NOT a prerequisite – it was
designed as an optional enrichment step but was blocking the main flow.
If provider is unavailable (edge case), the session stays at audit_passed.
"""

from app.models.session import SnapshotTrigger

try:
    await engine.snapshot(
        session_id=state.session_id,
        trigger=SnapshotTrigger.auto_audit_passed,
    )
except Exception as snap_exc:
    logger.warning(
        "Auto snapshot for audit_passed failed (session=%s): %s",
        state.session_id,
        snap_exc,
    )

# Clean up readiness metadata keys that are no longer needed.
new_meta = dict(state.metadata or {})
for k in (
    "__readiness_missing_codes",
    "__readiness_last_prompt",
    "__readiness_followup_count",
    "__readiness_nl_solution_direction",
    "__readiness_nl_solution_summary",
    "__readiness_nl_sector_acknowledged",
    "__gap_first_question",
):
    new_meta.pop(k, None)
state = state.model_copy(update={"metadata": new_meta})

# — Always auto-generate proposals when provider is available —————
if provider is not None:
    logger.info(
        "Session %s: audit_passed – auto-generating H1/H2/H3 proposals.",
        state.session_id,
    )
    try:
        return await self._generate_proposals(state, provider)
    except Exception as exc:
        logger.error(
            "Auto-generate proposals failed for session %s: %s – session
            stays at audit_passed.",
            state.session_id, exc,
        )
        # Return the cleaned-up state at audit_passed so the user can
        retry

    return state

```

```

        # Provider not available - session waits at audit_passed.
        logger.warning(
            "Session %s: audit_passed but no provider available - proposals cannot
be generated.",
            state.session_id,
        )
        return state

    async def _run_proposal_readiness_turn(
        self,
        state: SessionState,
        user_message: str,
        provider: Any,
        stream_cb: Callable[[str], Awaitable[None]],
    ) -> SessionState:
        """Collect clarifications until deterministic readiness rules pass."""
        from app.graph import get_graph_engine

        from app.core.proposal_readiness import (
            evaluate_readiness,
            readiness_followup_prompt,
        )
        from app.core.readiness_ingestion import ingest_readiness_nl

        # Ingest the user's NL reply - updates domain_chips, validated_metrics,
        # and __readiness_nl_solution_direction /
__readiness_nl_sector_acknowledged.
        state = await ingest_readiness_nl(state, user_message, provider)

        # If domain_chips were populated (by ingestion or prior turns), mark the
        # sector as NL-acknowledged so evaluate_readiness can relax the domain
        check.
        if state.domain_chips and not
state.metadata.get("__readiness_nl_sector_acknowledged"):
            meta_update = dict(state.metadata)
            meta_update["__readiness_nl_sector_acknowledged"] = True
            state = state.model_copy(update={"metadata": meta_update})

        engine = get_graph_engine()
        graph = await engine.get_current_graph(
            session_id=state.session_id,
            active_framework_id=state.active_framework_id,
        )
        followup_count = int(state.metadata.get("__readiness_followup_count", 0)
or 0)
        readiness = evaluate_readiness(state, graph)

        logger.info(

```

```

        "Readiness evaluation for session %s (attempt=%d): sufficient=%s
missing=%s",
        state.session_id,
        followup_count,
        readiness.is_sufficient,
        readiness.missing_codes,
    )

    if readiness.is_sufficient:
        passed = self._transition(state, SessionStatus.audit_passed)
        cleared = await self._apply_audit_passed_success(passed, engine,
provider=provider)
        meta_ok = dict(cleared.metadata)
        # Clean up readiness tracking keys
        for k in (
            "__readiness_followup_count",
            "__readiness_missing_codes",
            "__readiness_last_prompt",
            "__readiness_nl_sector_acknowledged",
            "__readiness_nl_solution_direction",
            "__readiness_nl_solution_summary",
        ):
            meta_ok.pop(k, None)
        return cleared.model_copy(update={"metadata": meta_ok})

    # Shorter follow-ups after the initial long message from run_audit (same
codes, new turn).
    attempt = followup_count + 1
    prompt = readiness_followup_prompt(readiness.missing_codes,
attempt=attempt)
    if prompt:
        await stream_cb(prompt)
    transcript = list(state.transcript)
    transcript.append(Message(role=MessageRole.assistant, content=prompt))
    meta = {
        **state.metadata,
        "__readiness_missing_codes": readiness.missing_codes,
        "__readiness_last_prompt": prompt,
        "__readiness_followup_count": followup_count + 1,
    }
    meta.pop("__gap_first_question", None)
    return state.model_copy(update={"transcript": transcript, "metadata":
meta})

# — Gap Analysis (Semana 6 / RF25) —————

async def _run_gap_analysis_turn(
    self,
    state: SessionState,
    user_message: str,

```

```

        provider: Any,
        stream_cb: Callable[[str], Awaitable[None]],
    ) -> SessionState:
        """Handle one user turn during the gap analysis phase (audit_passed).

        Flow:
        1. Parse the user response into a TargetMetric for the current metric.
        2. Validate the target (300% rule). If invalid, ask for justification.
        3. Persist the TargetMetric, compute coverage, emit WS coverage event.
        4. If all covered → call ProposalClassifier → transition to
        proposals_ready.
        5. Otherwise → ask for the next uncovered metric.
        """

        from app.core.gap_analysis_controller import GapAnalysisController

        if not state.validated_metrics:
            return await self._generate_proposals(state, provider)

        gap = GapAnalysisController(provider)

        # Determine which metric we are currently collecting
        covered_names = {t.metric_name for t in state.target_metrics}
        pending_metrics = [m for m in state.validated_metrics if m.name not in
        covered_names]

        # Pending warning check: user may be answering a justification request
        awaiting_justification_name: str | None = state.metadata.get(
            "__gap_awaiting_justification"
        )

        if awaiting_justification_name:
            # User provided justification for an over-threshold target
            pending_target_raw = state.metadata.get("__gap_pending_target")
            if pending_target_raw:
                from app.models.session import TargetMetric
                pending_target = TargetMetric(**pending_target_raw)
                pending_target = pending_target.model_copy(
                    update={"overridden": True, "justification": user_message}
                )
                new_targets = list(state.target_metrics) + [pending_target]
                new_metadata = {
                    k: v
                    for k, v in state.metadata.items()
                    if k not in ("__gap_awaiting_justification",
                    "__gap_pending_target")
                }
                state = state.model_copy(
                    update={"target_metrics": new_targets, "metadata":
                    new_metadata}
                )

```

```

        # Re-compute pending after accepting override
        covered_names = {t.metric_name for t in state.target_metrics}
        pending_metrics = [m for m in state.validated_metrics if m.name
not in covered_names]
        elif pending_metrics:
            current_metric = pending_metrics[0]
            target = await gap.parse_target_from_response(user_message,
current_metric)
            if target is not None:
                is_valid, warning_msg = gap.validate_target(current_metric,
target)

                if not is_valid:
                    # Emit warning via stream_cb, store pending target in metadata
                    await stream_cb(warning_msg)
                    new_metadata = {
                        **state.metadata,
                        "__gap_awaiting_justification": current_metric.name,
                        "__gap_pending_target": target.model_dump(mode="json"),
                    }
                    state = state.model_copy(update={"metadata": new_metadata})
                    return state

                # Valid target → persist
                new_targets = list(state.target_metrics) + [target]
                state = state.model_copy(update={"target_metrics": new_targets})
                covered_names = {t.metric_name for t in state.target_metrics}
                pending_metrics = [m for m in state.validated_metrics if m.name
not in covered_names]
            else:
                # Could not parse – re-ask the same metric
                question = gap.generate_next_question(state.validated_metrics,
state.target_metrics)
                if question:
                    await stream_cb(question)
                    return state

        # Compute & emit coverage
        coverage = gap.compute_coverage(state.validated_metrics,
state.target_metrics)

        if coverage >= 1.0 and state.validated_metrics:
            # All targets collected → generate proposals
            state = await self._generate_proposals(state, provider)
        else:
            # Ask for the next uncovered metric
            next_question = gap.generate_next_question(state.validated_metrics,
state.target_metrics)
            if next_question:
                await stream_cb(next_question)

```

```

        # WS coverage event will be emitted via __gap_coverage_update key picked
        up in session.py
        state = state.model_copy(
            update={"metadata": {**state.metadata, "__gap_coverage_update":
coverage}}}
        )
        return state

    async def _generate_proposals(
        self,
        state: SessionState,
        provider: Any,
    ) -> SessionState:
        """Call ProposalClassifier + KBRetriever and transition to
        proposals_ready."""
        from app.core.proposal_classifier import ProposalClassifier
        from app.graph import get_graph_engine
        from app.models.session import SnapshotTrigger, ProposalKBCase

        engine = get_graph_engine()
        graph_summary_text = await engine.get_graph_summary_text(state.session_id)

        classifier = ProposalClassifier(provider)
        try:
            proposal_set = await classifier.generate_proposals(state,
graph_summary_text)
        except Exception as exc:
            logger.error(
                "ProposalClassifier failed for session %s: %s", state.session_id,
exc
            )
            return state

    # — KB Evidence enrichment —————
    try:
        from app.kb.retriever import KBRetriever
        session_domain = None
        if state.domain_chips:
            session_domain = state.domain_chips[0]

        # The provider must support embedding for KBRetriever
        if hasattr(provider, "embed"):
            retriever = KBRetriever(embedder=provider)
            for card in (proposal_set.h1, proposal_set.h2, proposal_set.h3):
                try:
                    matches = await retriever.retrieve(
                        query=card.narrative,
                        session_domain=session_domain,
                        filters={"horizon": card.horizon},
                        top_k=2,

```

```

        session_id=state.session_id,
    )
    card.kb_cases = [
        ProposalKBCase(
            case_id=str(m.doc_id),
            title=f"{m.document.sector} -
{m.document.solution_type}",
            similarity_score=round(m.final_score, 3),
            sector=m.document.sector,
            domain=m.document.domain,
            horizon=m.document.horizon, # type:
ignore[arg-type]
            summary=(m.document.solution_description or
""[:300]),
        )
        for m in matches
    ]
except Exception as kb_card_exc:
    logger.warning(
        "KB retrieval failed for %s card: %s", card.horizon,
kb_card_exc
    )
except Exception as kb_exc:
    logger.warning("KB evidence enrichment failed: %s", kb_exc)

# Persist desired_state_value on matching graph nodes
try:
    for tm in state.target_metrics:
        await engine.update_node_desired_state(
            session_id=state.session_id,
            metric_name=tm.metric_name,
            desired_state_value=f"{tm.target_value} {tm.target_unit}
({tm.time_horizon})",
        )
except Exception as exc:
    logger.warning("Could not update desired_state_value on nodes: %s",
exc)

# Snapshot
try:
    await engine.snapshot(state.session_id,
SnapshotTrigger.proposals_generated)
except Exception as exc:
    logger.warning("proposals_generated snapshot failed: %s", exc)

state = self._transition(state, SessionStatus.proposals_ready)
state = state.model_copy(update={"proposals": proposal_set})
state = state.model_copy(
    update={"metadata": {**state.metadata, "__proposals_payload":
proposal_set.model_dump(mode="json")}}

```

```

    )
    return state

def get_session(self, session_id: str) -> SessionState:
    """Return the current SessionState for a session.

    Args:
        session_id: UUID of the session.

    Returns:
        The current SessionState.

    Raises:
        ValueError: If the session does not exist.
    """
    return self._load_state(session_id)

async def _run_audit_graph_node(self, raw_state: dict[str, Any]) -> dict[str, Any]:
    """LangGraph adapter: execute audit from a raw graph-node state dict."""
    recheck_node_ids = raw_state.get("__recheck_node_ids")
    recheck_list = (
        [str(node_id) for node_id in recheck_node_ids]
        if isinstance(recheck_node_ids, list)
        else None
    )
    # Provider may be injected into the raw state for auto-proposal
    generation.
    provider = raw_state.get("__provider")
    state = _dict_to_state(raw_state)
    audited = await self._execute_audit(
        state=state,
        recheck_node_ids=recheck_list,
        provider=provider,
    )
    return _state_to_dict(audited)

@staticmethod
def _delta_has_content(delta: GraphDelta) -> bool:
    return bool(
        delta.added_nodes
        or delta.updated_nodes
        or delta.removed_node_ids
        or delta.added_edges
        or delta.updated_edges
        or delta.removed_edge_ids
    )

@staticmethod
def _changed_node_ids_from_delta(delta: GraphDelta) -> set[str]:

```

```

        changed = {node.id for node in delta.added_nodes}
        changed.update(node.id for node in delta.updated_nodes)
        changed.update(delta.removed_node_ids)
        for edge in delta.added_edges + delta.updated_edges:
            changed.add(edge.source)
            changed.add(edge.target)
        return changed

    @staticmethod
    def _collect_direct_dependents(graph: "Any", changed_node_ids: set[str]) ->
list[str]:
        dependents = set()
        for edge in graph.edges:
            if edge.source in changed_node_ids:
                dependents.add(edge.target)
        return sorted(dependents)

    @staticmethod
    def _compute_structural_diff(pre_graph: "Any", post_graph: "Any") ->
GraphDelta:
        pre_nodes = {node.id: node for node in pre_graph.nodes}
        post_nodes = {node.id: node for node in post_graph.nodes}
        pre_edges = {edge.id: edge for edge in pre_graph.edges}
        post_edges = {edge.id: edge for edge in post_graph.edges}

        added_nodes = [post_nodes[nid] for nid in post_nodes.keys() -
pre_nodes.keys()]
        removed_node_ids = sorted(pre_nodes.keys() - post_nodes.keys())
        updated_nodes: list[Any] = []
        for nid in sorted(pre_nodes.keys() & post_nodes.keys()):
            a = pre_nodes[nid]
            b = post_nodes[nid]
            if (
                a.label != b.label
                or a.type != b.type
                or a.status != b.status
                or a.is_critical_path != b.is_critical_path
                or a.framework_zone != b.framework_zone
                or a.desired_state_value != b.desired_state_value
                or [m.model_dump(mode="json") for m in a.metrics] !=
[m.model_dump(mode="json") for m in b.metrics]
            ):
                updated_nodes.append(b)

        added_edges = [post_edges[eid] for eid in post_edges.keys() -
pre_edges.keys()]
        removed_edge_ids = sorted(pre_edges.keys() - post_edges.keys())
        updated_edges: list[Any] = []
        for eid in sorted(pre_edges.keys() & post_edges.keys()):
            a = pre_edges[eid]

```

```

        b = post_edges[eid]
        if (
            a.source != b.source
            or a.target != b.target
            or a.status != b.status
            or a.is_lateral_integration != b.is_lateral_integration
        ):
            updated_edges.append(b)

    return GraphDelta(
        added_nodes=added_nodes,
        updated_nodes=updated_nodes,
        removed_node_ids=removed_node_ids,
        added_edges=added_edges,
        updated_edges=updated_edges,
        removed_edge_ids=removed_edge_ids,
    )

    async def _execute_audit(
        self,
        state: SessionState,
        recheck_node_ids: list[str] | None = None,
        provider: Any = None,
        progress_cb: "Callable[[dict], Awaitable[None]] | None" = None,
    ) -> SessionState:
        """Execute causal audit against Neo4j graph and apply status
        transitions."""

        async def _noop(_: dict) -> None:
            return None

        emit = progress_cb or _noop

        if state.status not in {
            SessionStatus.exploration_complete,
            SessionStatus.audit_pending,
            SessionStatus.audit_passed,
            SessionStatus.proposal_inputs_pending,
        }:
            raise InvalidTransitionError(state.status,
            SessionStatus.audit_pending)

        entry_status = state.status

        if entry_status == SessionStatus.exploration_complete:
            state = self._transition(state, SessionStatus.audit_pending)
        elif entry_status == SessionStatus.proposal_inputs_pending:
            state = self._transition(state, SessionStatus.audit_pending)

        can_do_incremental = bool(

```

```

        recheck_node_ids
        and state.audit_findings
        and all(f.affected_node_ids for f in state.audit_findings)
    )
    effective_recheck_ids = list(recheck_node_ids or []) if can_do_incremental
else []

    transcript_messages = [m.content for m in state.transcript]
    from app.graph import get_graph_engine

    from app.core.proposal_readiness import evaluate_readiness,
    readiness_prompt_from_result

    engine = get_graph_engine()

    await emit({
        "type": "audit_progress",
        "step": "graph_check",
        "message": "Verificando consistencia causal del grafo...",
    })
    fresh_findings = await engine.detect_causal_issues(
        session_id=state.session_id,
        transcript_messages=transcript_messages,
    )
    merged_findings = self._merge_audit_findings(
        existing=state.audit_findings,
        fresh=fresh_findings,
        recheck_node_ids=effective_recheck_ids or None,
    )
    has_blocking = any(f.is_blocking for f in merged_findings)

    updated = state.model_copy(update={"audit_findings": merged_findings})

    if has_blocking:
        return updated.model_copy(update={"status":
        SessionStatus.audit_pending})

    if entry_status == SessionStatus.audit_passed:
        # Re-audit while already passed: refresh findings, then attempt
        proposal generation.
        refreshed = updated.model_copy(update={"status":
        SessionStatus.audit_passed})
        return await self._apply_audit_passed_success(refreshed, engine,
        provider=provider)

    shell = updated.model_copy(update={"status": SessionStatus.audit_pending})

    await emit({
        "type": "audit_progress",
        "step": "readiness",

```

```

        "message": "Evaluando completitud de la información del diagnóstico...",
    })
    graph = await engine.get_current_graph(
        session_id=shell.session_id,
        active_framework_id=shell.active_framework_id,
    )
    readiness = evaluate_readiness(shell, graph)

    if not readiness.is_sufficient:
        prompt = readiness_prompt_from_result(readiness)
        meta = {
            **shell.metadata,
            "__readiness_missing_codes": readiness.missing_codes,
            "__readiness_last_prompt": prompt,
            "__readiness_followup_count": 0,
        }
        meta.pop("__gap_first_question", None)
        # NOTE: do NOT pop __readiness_nl_solution_direction here - the user
        # may have described a solution direction during a prior chat turn
        # and we must not erase that context on every re-audit.
        pending = self._transition(shell,
SessionStatus.proposal_inputs_pending)
        pending = pending.model_copy(update={"metadata": meta})
        return pending.model_copy(
            update={
                "transcript": [
                    *pending.transcript,
                    Message(role=MessageRole.assistant, content=prompt),
                ]
            }
        )

    passed = self._transition(shell, SessionStatus.audit_passed)
    return await self._apply_audit_passed_success(passed, engine,
provider=provider)

@staticmethod
def _merge_audit_findings(
    existing: list[AuditFinding],
    fresh: list[AuditFinding],
    recheck_node_ids: list[str] | None,
) -> list[AuditFinding]:
    """Merge findings for full or incremental re-audit flows."""
    if not recheck_node_ids:
        return SessionController._dedupe_findings(fresh)

    recheck_set = set(recheck_node_ids)
    retained = [
        finding
        for finding in existing

```

```

        if not finding.affected_node_ids
        or recheck_set.isdisjoint(set(finding.affected_node_ids))
    ]
    return SessionController._dedupe_findings([*retained, *fresh])

@staticmethod
def _dedupe_findings(findings: list[AuditFinding]) -> list[AuditFinding]:
    seen: set[tuple[str, bool, tuple[str, ...], str]] = set()
    deduped: list[AuditFinding] = []
    for finding in findings:
        key = (
            finding.rule,
            bool(finding.is_blocking),
            tuple(sorted(finding.affected_node_ids)),
            finding.description.strip().lower(),
        )
        if key in seen:
            continue
        seen.add(key)
        deduped.append(finding)
    return deduped

# — Private helpers —————

def _load_state(self, session_id: str) -> SessionState:
    """Load SessionState from PostgreSQL by session_id."""
    db_session = (
        self._db.query(DBSession)
        .filter(DBSession.id == uuid.UUID(session_id))
        .first()
    )
    if db_session is None:
        raise ValueError(f"Session not found: {session_id}")

    transcript: list[Message] = []
    if db_session.transcript:
        raw: list[dict] = db_session.transcript # type: ignore[assignment]
        transcript = [Message(**m) for m in raw]

    imo_scores: dict[str, float] = {}
    if db_session.imo_scores:
        imo_scores = dict(db_session.imo_scores) # type: ignore[arg-type]

    validated_metrics = []
    if db_session.validated_metrics:
        from app.models.session import Metric
        raw_metrics: list[dict] = db_session.validated_metrics # type:
ignore[assignment]
        validated_metrics = [Metric(**m) for m in raw_metrics]
    audit_findings: list[AuditFinding] = []

```

```

        if db_session.audit_findings:
            raw_findings: list[dict[str, Any]] = db_session.audit_findings #
type: ignore[assignment]
            for finding in raw_findings:
                try:
                    audit_findings.append(AuditFinding(**finding))
                except Exception as finding_exc:
                    logger.warning(
                        "Skipping invalid persisted audit finding in session %s:
%s",
                        session_id,
                        finding_exc,
                    )
            active_framework_raw = db_session.active_framework if
isinstance(db_session.active_framework, dict) else None
            framework_restructure_cache = {}
            metadata: dict[str, Any] = {}
            if active_framework_raw:
                _raw_cache = active_framework_raw.get("__framework_restructure_cache",
{})

                if isinstance(_raw_cache, dict):
                    framework_restructure_cache = _raw_cache
                _raw_metadata = active_framework_raw.get("__metadata", {})
                if isinstance(_raw_metadata, dict):
                    metadata = _raw_metadata
            active_framework_clean = None
            if active_framework_raw:
                active_framework_clean = {
                    k: v
                    for k, v in active_framework_raw.items()
                    if not str(k).startswith("__")
                } or None

# — Gap Analysis state —————
from app.models.session import TargetMetric, ProposalSet
target_metrics = []
try:
    raw_targets_raw = db_session.target_metrics
    if raw_targets_raw:
        raw_targets: list[dict] = raw_targets_raw # type:
ignore[assignment]
        for t in raw_targets:
            try:
                target_metrics.append(TargetMetric(**t))
            except Exception as _te:
                logger.warning("Skipping invalid target_metric: %s", _te)
except Exception as _col_exc:
    logger.debug("target_metrics column not available: %s", _col_exc)

proposals = None

```

```

try:
    raw_proposals = db_session.proposals
    if raw_proposals and isinstance(raw_proposals, dict):
        try:
            proposals = ProposalSet(**raw_proposals)
        except Exception as _pe:
            logger.warning("Could not deserialize proposals for session
%s: %s", session_id, _pe)
    except Exception as _col_exc:
        logger.debug("proposals column not available: %s", _col_exc)

    domain_chips: list[str] = []
    raw_dc = getattr(db_session, "domain_chips", None)
    if isinstance(raw_dc, list):
        domain_chips = [str(x) for x in raw_dc]

    return SessionState(
        session_id=str(db_session.id),
        status=SessionStatus(db_session.status),
        transcript=transcript,
        imo_scores=imo_scores,
        validated_metrics=validated_metrics,
        domain_chips=domain_chips,
        active_framework_id=db_session.active_framework_id, # type:
ignore[arg-type]
        active_framework=active_framework_clean, # type: ignore[arg-type] -
loaded as dict
        # graph_initialized is stored as a boolean inside the JSONB
active_framework dict
        # under the key "__graph_initialized". Fallback: infer from
active_framework_id
        # for sessions created before this flag was introduced.
        graph_initialized=(
            bool(
                isinstance(active_framework_raw, dict)
                and active_framework_raw.get("__graph_initialized", False)
            )
            or bool(db_session.active_framework_id) # backward-compat
inference
        ),
        graph_revision=int(getattr(db_session, "graph_revision", 0) or 0),
        structural_revision=int(getattr(db_session, "structural_revision", 0)
or 0),

        solution_layers=self._hydrate_solution_layers(db_session),
        framework_restructure_cache=framework_restructure_cache,
        audit_findings=audit_findings,
        target_metrics=target_metrics,
        gap_analysis_metric_index=int(len(target_metrics)),
        proposals=proposals,
        metadata=metadata,

```

```

    )

    @staticmethod
    def _hydrate_solution_layers(db_session) -> "SolutionLayerSet":
        """Load SolutionLayerSet from DB JSONB with graceful fallback."""
        from app.models.session import SolutionLayerSet
        raw = getattr(db_session, "solution_layers", None)
        if not raw or not isinstance(raw, dict):
            return SolutionLayerSet()
        try:
            return SolutionLayerSet(**raw)
        except Exception:
            return SolutionLayerSet()

    def _persist_state(self, session_id: str, state: SessionState) -> None:
        """Persist the current SessionState to PostgreSQL."""
        db_session = (
            self._db.query(DBSession)
            .filter(DBSession.id == uuid.UUID(session_id))
            .first()
        )
        if db_session is None:
            raise ValueError(f"Session not found for persistence: {session_id}")

        db_session.status = state.status.value # type: ignore[assignment]
        db_session.transcript = [ # type: ignore[assignment]
            {"role": m.role.value if hasattr(m.role, 'value') else m.role,
             "content": m.content}
            for m in state.transcript
        ]
        db_session.imo_scores = state.imo_scores # type: ignore[assignment]
        db_session.validated_metrics = [ # type: ignore[assignment]
            m.model_dump() for m in state.validated_metrics
        ]
        db_session.audit_findings = [ # type: ignore[assignment]
            f.model_dump(mode="json") for f in state.audit_findings
        ]
        db_session.domain_chips = state.domain_chips # type: ignore[assignment]
        # Gap Analysis persistence (Paso 6.1-6.3 - columns added by migration
        # a7f3b1c8d2e5)
        try:
            db_session.target_metrics = [ # type: ignore[assignment]
                t.model_dump(mode="json") for t in state.target_metrics
            ]
        except Exception as _col_exc:
            logger.debug("Could not persist target_metrics (column may not exist
            yet): %s", _col_exc)
        try:
            if state.proposals is not None:

```

```

        db_session.proposals = state.proposals.model_dump(mode="json") #
type: ignore[assignment]
    except Exception as _col_exc:
        logger.debug("Could not persist proposals (column may not exist yet):
%s", _col_exc)
        db_session.active_framework_id = state.active_framework_id # type:
ignore[assignment]
        db_session.graph_revision = int(state.graph_revision or 0) # type:
ignore[assignment]
        # Structural revision persistence
        try:
            db_session.structural_revision = int(getattr(state,
"structural_revision", 0) or 0) # type: ignore[assignment]
        except Exception:
            pass # Column may not exist yet before migration
        # Solution layers persistence
        try:
            _sl = getattr(state, "solution_layers", None)
            if _sl is not None:
                db_session.solution_layers = ( # type: ignore[assignment]
                    _sl if isinstance(_sl, dict) else _sl.model_dump(mode="json")
                )
        except Exception:
            pass # Column may not exist yet before migration
        # Serialize FrameworkInfo → dict for JSONB storage.
        # We piggyback the graph_initialized flag inside this JSONB dict to avoid
        # a new DB migration. It is stored under the key "__graph_initialized".
        _fw_dict: dict | None = None
        if state.active_framework is None:
            _fw_dict = {
                "__graph_initialized": state.graph_initialized,
                "__framework_restructure_cache":
state.framework_restructure_cache,
                "__metadata": state.metadata,
            }
        elif isinstance(state.active_framework, dict):
            _fw_dict = {
                **state.active_framework,
                "__graph_initialized": state.graph_initialized,
                "__framework_restructure_cache":
state.framework_restructure_cache,
                "__metadata": state.metadata,
            }
        else:
            _fw_dict = {
                **state.active_framework.model_dump(),
                "__graph_initialized": state.graph_initialized,
                "__framework_restructure_cache":
state.framework_restructure_cache,
                "__metadata": state.metadata,

```

```

    }
    db_session.active_framework = _fw_dict # type: ignore[assignment]

    self._db.commit()
    logger.debug(
        "Session persisted: %s (status=%s, graph_initialized=%s,
graph_revision=%s)",
        session_id,
        state.status,
        state.graph_initialized,
        state.graph_revision,
    )

def _transition(
    self,
    state: SessionState,
    target: SessionStatus,
) -> SessionState:
    """Validate and apply a state transition.

    Args:
        state: Current SessionState.
        target: Desired next status.

    Returns:
        Updated SessionState with the new status.

    Raises:
        InvalidTransitionError: If the transition is not allowed.
    """
    allowed = _VALID_TRANSITIONS.get(state.status, set())
    if target not in allowed:
        raise InvalidTransitionError(
            current=state.status,
            target=target,
        )
    logger.info(
        "Session %s: %s → %s",
        state.session_id,
        state.status,
        target,
    )
    return state.model_copy(update={"status": target})

```

— Exceptions

```

class InvalidTransitionError(Exception):
    """Raised when an illegal session state transition is attempted."""

```

```

def __init__(self, current: SessionStatus, target: SessionStatus) -> None:
    self.current = current
    self.target = target
    super().__init__(
        f"Invalid transition: {current!r} → {target!r}. "
        f"Allowed from {current!r}: {sorted(_VALID_TRANSITIONS.get(current,
set()))}"
    )

# — State serialization helpers —————

def _state_to_dict(state: SessionState) -> dict[str, Any]:
    """Convert SessionState to a plain dict for LangGraph consumption."""
    return {
        "session_id": state.session_id,
        "status": state.status,
        "transcript": [{"role": m.role, "content": m.content} for m in
state.transcript],
        "imo_scores": state.imo_scores,
        "validated_metrics": [m.model_dump() for m in state.validated_metrics],
        "analyst_output": state.analyst_output,
        "audit_findings": [f.model_dump() for f in state.audit_findings],
        "canvas_delta": state.canvas_delta,
        "active_framework_id": state.active_framework_id,
        "active_framework": state.active_framework,
        "framework_selected_by": state.framework_selected_by,
        "framework_change_history": [c.model_dump() for c in
state.framework_change_history],
        "restructure_output": state.restructure_output,
        "graph_revision": state.graph_revision,
        "structural_revision": state.structural_revision,
        "solution_layers": state.solution_layers.model_dump(mode="json") if
state.solution_layers else {},
        "framework_restructure_cache": state.framework_restructure_cache,
        "metadata": state.metadata,
        "graph_initialized": state.graph_initialized,    # passed through LangGraph
dict
        "domain_chips": state.domain_chips,
    }

def _dict_to_state(raw: dict[str, Any]) -> SessionState:
    """Reconstruct SessionState from a LangGraph output dict."""
    from app.models.session import FrameworkChange, GraphRestructureOutput, Metric

    transcript = [Message(**m) for m in raw.get("transcript", [])]
    validated_metrics = [Metric(**m) for m in raw.get("validated_metrics", [])]

```

```

audit_findings = [AuditFinding(**f) for f in raw.get("audit_findings", [])]
framework_changes = [
    c if isinstance(c, FrameworkChange) else FrameworkChange(**c)
    for c in raw.get("framework_change_history", [])
]
raw_restructure = raw.get("restructure_output")
restructure_output = (
    raw_restructure
    if isinstance(raw_restructure, GraphRestructureOutput) or raw_restructure
is None
    else GraphRestructureOutput(**raw_restructure)
)

return SessionState(
    session_id=raw["session_id"],
    status=SessionStatus(raw["status"]),
    transcript=transcript,
    imo_scores=raw.get("imo_scores", {}),
    validated_metrics=validated_metrics,
    analyst_output=raw.get("analyst_output"),
    audit_findings=audit_findings,
    canvas_delta=raw.get("canvas_delta"),
    active_framework_id=raw.get("active_framework_id"),
    active_framework=raw.get("active_framework"),
    framework_selected_by=raw.get("framework_selected_by"),
    framework_change_history=framework_changes,
    restructure_output=restructure_output,
    graph_revision=raw.get("graph_revision", 0),
    structural_revision=raw.get("structural_revision", 0),
    framework_restructure_cache=raw.get("framework_restructure_cache", {}),
    metadata=raw.get("metadata", {}),
    graph_initialized=raw.get("graph_initialized", False), # propagated from
node dict
    domain_chips=raw.get("domain_chips", []),
)

```